SPARQL: запросы к графу знаний

Туториал на графе «Евгения Онегина»

О. М. Атаева А. П. Халов

Осень 2026

Курс «От интеллектуального поиска к интеллектуальным агентам», ВМК МГУ
Блокнот к туториалу: `sparql_tutorial.ipynb`

Вопрос, на который поиск по словам не отвечает

Вопрос из выборки занятия 5: «ответ Онегина на письмо Татьяны». Нужны строфы объяснения в саду, глава 4, строфы XII–XVI.

Поиск по словам, BM25F

ранг	строфа	нужна?
1	8.XXXIII	нет
2	3.XXXVI	нет
3	3.ПТ, письмо Татьяны	нет
4	3.XXXI	нет
5	8.XXI	нет

Слова запроса стоят в самом письме и в строфах о нём. В строфах ответа их почти нет.

Запрос к графу

```
SELECT ?строфа ?перваяСтрока WHERE {
    ?ответ  :ответНа  ?письмо .
    ?письмо :автор    :Татьяна .
    ?ответ  :вСтрофах ?с .
    ?с rdfs:label ?строфа ; :перваяСтрока ?перваяСтрока .
} ORDER BY ?строфа
```

4.XII	Минуты две они молчали,
4.XIII	Когда бы жизнь домашним кругом
4.XIV	Но я не создан для блаженства;
4.XV	Что может быть на свете хуже
4.XVI	Мечтам и годам нет возврата;

Главное

Поиск по словам находит строфу по её словам. Граф находит строфу по связи «ответ на». Связи записаны в графе, и запрашивает их язык SPARQL.

Что умеет запрос к графу



Что даёт граф

- факты, соединённые через несколько рёбер;
- цепочки любой длины: причины, зависимости;
- счёт и группировку по связям;
- проверку утверждения: есть такой факт или нет.

Где это в курсе

- модуль 1: онтология и граф положения о рецензировании;
- лекция 7: граф как канал поиска;
- лекция 8: проверка ответа модели запросом к графу;
- лекция 11: сервер MCP над графом.

Главное

SPARQL есть язык, на котором агент задаёт графу вопрос. Туториал вводит его основные понятия на одном знакомом графе.

План туториала

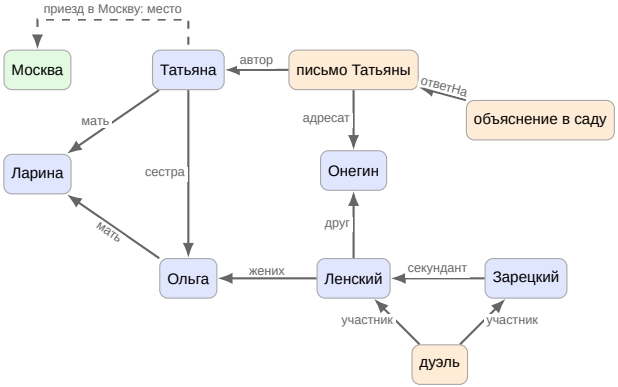
блок	понятия	шаги блокнота
граф	тройка, IRI, литерал, запись Turtle, две части графа	0–1
первые запросы	`SELECT`, `WHERE`, переменная, шаблон из нескольких троек, `FILTER`, язык метки	2–4
сложные шаблоны	`OPTIONAL`, `UNION`, `BIND`, `FILTER NOT EXISTS`, `MINUS`, агрегаты	5–8
пути	`/` `^` `|` `+` `*`, классы и подсвойства без вывода, окрестность	9–11
другие формы ответа	`ASK`, `CONSTRUCT`, именованные графы	12–13
граф в работе	граф и поиск по словам, положение о рецензировании, точка доступа, инструмент агента	14–17

Каждый запрос со слайда есть в блокноте `sparql_tutorial.ipynb` и выполняется на `rdflib`. Результаты на слайдах взяты из прогона блокнота.

Главное

Туториал идёт от одной тройки к запросам, которые агент задаёт графу в работе.

Граф «Онегина»: что в нём



Фрагмент ручной разметки. Синие узлы суть персонажи, оранжевые события, зелёные места.

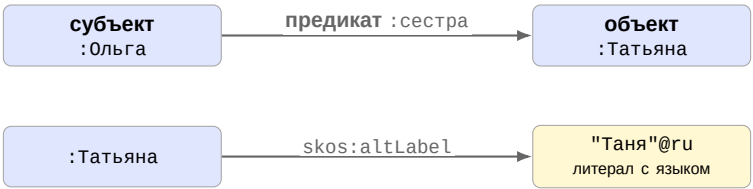
что	сколько
персонажи	11
места	3
события	12
строфы	360
упоминания героев в строфах	289
троек ручной разметки	269
троек из текста	2453

Строфа обозначается «глава.строфа»: 4.XII есть глава 4, строфа XII. Письма обозначены 3.ПТ и 8.ПО.

Главное

Граф построен из того же корпуса, что и индекс занятия 5. Поэтому на одних вопросах можно сравнить оба канала.

Тройка: субъект, предикат, объект



- **Граф RDF** есть множество троек. Порядка у троек нет, повторов нет.
- **IRI** есть глобальное имя: `http://example.org/onegin#Татьяна`. Префикс сокращает его до `:Татьяна`.
- **Литерал** есть значение: строка, число, дата. У строки бывает метка языка `@ru`, у числа тип `xsd:integer`.
- Литерал стоит только на месте объекта. Из литерала ребро не выходит.

Главное

Одна тройка есть одно ребро графа. Запрос SPARQL ищет в графе рёбра, которые подходят под шаблон.

Запись Turtle

```
@prefix :     <http://example.org/onegin#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix skos: <http://www.w3.org/2004/02/skos/core#> .

:Татьяна a :ГлавныйГерой ;
    rdfs:label "Татьяна Ларина"@ru ;
    :живёт :Деревня, :Петербург ;
    :мать :Ларина ;
    :муж :Князь ;
    :няня :Филипьевна ;
    :сестра :Ольга ;
    skos:altLabel "Таня"@ru, "Татьяна"@ru .
```

- `@prefix` задаёт сокращение для начала IRI.
- a значит `rdf:type`: Татьяна есть главный герой.
- **точка с запятой** продолжает тот же субъект;
- **запятая** продолжает тот же субъект и предикат: две тройки `:живёт`.
- **точка** закрывает описание субъекта.

Двенадцать строк слева дают двенадцать троек.

Главное

Turtle есть запись графа для человека. Шаблоны SPARQL пишутся в той же записи, поэтому её стоит читать свободно.

Две части графа

ручная разметка: g:relations

Персонажи, места, события. Родство и дружба, участники событий, «привело к», «ответ на», порядок событий. Классы и иерархия свойств.

Файл `data/onegin_relations.ttl`, 269 троек.

извлечено из текста: g:text

360 строф: глава, номер, первая строка, текст. Тройки «герой упомянут в строфе», найденные программой по формам имени.

Файл `data/onegin_text.ttl`, 2453 тройки.

герой	формы имени, по которым находится упоминание
Татьяна	Татьяна, Татьяне, Татьяной, Таня, Тани, Таню
Ленский	Ленский, Ленской, Ленского, Владимир
Онегин	Онегин, Онегина, Евгений, Евгения

Ларина и дядя Онегина из текста не извлекаются: в главе 8 «Лариной» названа Татьяна.

Главное

У каждой части своё происхождение и своя надёжность. В наборе данных они лежат как два именованных графа, и запрос может спросить, откуда факт.

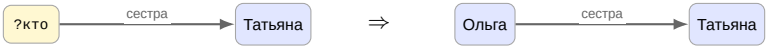
Первый запрос

```
PREFIX : <http://example.org/onegin#>

SELECT ?кто
WHERE {
    ?кто :сестра :Татьяна .
}
```

кто
:Ольга

- `PREFIX` задаёт сокращения, как `@prefix` в Turtle.
- `SELECT` перечисляет переменные ответа.
- `WHERE` содержит **шаблон**: тройку, где на месте неизвестного стоит **переменная** `?кто`.
- Машина запросов ищет все тройки, подходящие под шаблон. Тройка `:Ольга :сестра :Татьяна` подходит при `?кто = :Ольга`.



Главное

Запрос есть граф с дырами. Ответ состоит из всех способов заполнить дыры так, чтобы шаблон совпал с куском графа.

Переменная на месте свойства

```
SELECT ?свойство ?значение WHERE {
    :Татьяна ?свойство ?значение .
} LIMIT 12
```

Переменная может стоять на любом из трёх мест тройки. Здесь неизвестны и свойство, и значение. Запрос перечисляет всё, что граф знает о Татьяне. `LIMIT` оставляет первые двенадцать строк из 112.

свойство	значение
rdf:type	:ГлавныйГерой
rdfs:label	Татьяна Ларина
:живёт	:Деревня
:живёт	:Петербург
:мать	:Ларина
:муж	:Князь
:няня	:Филипьевна
:сестра	:Ольга
skos:altLabel	Таня
skos:altLabel	Татьяна
:упомянут	:строфа_2_XXIV
:упомянут	:строфа_2_XXV

Главное

Запрос со свойством-переменной есть способ осмотреться в незнакомом графе: какие у узла рёбра и куда они ведут.

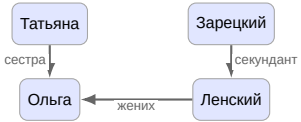
Шаблон из нескольких троек

Кто был секундантом у жениха сестры Татьяны?

```
SELECT ?сестра ?жених ?секундант WHERE {
    :Татьяна   :сестра    ?сестра .
    ?жених     :жених     ?сестра .
    ?секундант :секундант ?жених .
}
```

сестра	жених	секундант
:Ольга	:Ленский	:Зарецкий

Несколько троек образуют **базовый шаблон графа**. Общая переменная соединяет тройки: значение `?сестра` в первой тройке совпадает со значением во второй. Это соединение, как `JOIN` в SQL. Порядок троек ответа не меняет.



Главное

Запрос прошёл три ребра. В романе эти три факта стоят в разных главах, и поиск по словам их не соединит.

Литералы и язык метки

```
SELECT ?герой WHERE {
    ?герой skos:altLabel "Таня" .
}
```

Ответ пуст.

```
SELECT ?герой ?метка WHERE {
    ?герой skos:altLabel "Таня"@ru ;
           rdfs:label ?метка .
}
```

герой	метка
:Татьяна	Татьяна Ларина

Строка `"Таня"` и строка `"Таня"@ru` суть разные значения. В графе записана вторая, и запрос без метки языка её не находит.

Так агент находит узел по имени из вопроса: имя сравнивается с `rdfs:label` и `skos:altLabel`. Это простейшее связывание сущностей, подробно оно разбирается в лекции 7.

Главное

Самая частая причина пустого ответа есть несовпадение литерала: метка языка, регистр, лишний пробел.

FILTER: условие на решение

```
SELECT ?герой ?имя WHERE {
    ?герой skos:altLabel ?имя .
    FILTER(STRSTARTS(LCASE(STR(?имя)), "та"))
}
```

герой	имя
:Татьяна	Таня
:Татьяна	Татьяна

```
SELECT ?событие ?глава WHERE {
    ?событие :глава ?глава ; a ?класс .
    FILTER(?глава >= 6 && ?класс != :Строфа)
} ORDER BY ?глава ?событие
```

событие	глава
:вызов	6
:гибель_Ленского	6
:дуэль	6
:приезд_в_Москву	7
:встреча_в_Петербурге	8
:письмо_Онегина	8
:последнее_объяснение	8

`FILTER` оставляет решения, для которых условие истинно. Внутри: сравнения, `&&`, `||`, `!` и функции над строками `STR`, `LCASE`, `STRSTARTS`, `CONTAINS`, `REGEX`. `ORDER BY` сортирует ответ.

Главное

Шаблон находит кандидатов, FILTER отбрасывает лишних. У строф тоже есть номер главы, поэтому они исключены условием на класс.

OPTIONAL: необязательная часть

```
SELECT ?герой ?муж WHERE {
    ?герой a :ГлавныйГерой .
    OPTIONAL { ?герой :муж ?муж }
}
```

герой	муж
:Ольга	
:Ленский	
:Татьяна	:Князь
:Онегин	

Без `OPTIONAL` шаблон требует, чтобы совпали обе тройки, и в ответе осталась бы одна Татьяна.

С `OPTIONAL` строка остаётся, когда необязательная часть не нашлась. Переменная `?муж` в такой строке пуста.

Главное

OPTIONAL нужен, когда у одних узлов свойство есть, у других нет. Так собирают карточку сущности с неполными данными.

UNION и BIND: связи в обе стороны

```
SELECT ?направление ?связь ?кто WHERE {
  { :Ленский ?связь ?кто .
    BIND("от Ленского" AS ?направление) }
  UNION
  { ?кто ?связь :Ленский .
    BIND("к Ленскому" AS ?направление) }
  ?кто a ?класс .
  ?класс rdfs:subClassOf :Персонаж .
}
```

направление	связь	кто
от Ленского	:друг	:Онегин
от Ленского	:жених	:Ольга
к Ленскому	:секундант	:Зарецкий
к Ленскому	:друг	:Онегин

Ребро записано в одну сторону. Зарецкий секундант у Ленского, и это ребро идёт к Ленскому. `UNION` собирает решения обоих шаблонов. `BIND` записывает значение в новую переменную.

Главное

Направление ребра выбрал тот, кто строил граф. Запрос, которому направление безразлично, перечисляет обе стороны через UNION.

Отсутствие и открытый мир

Какие персонажи не упомянуты ни в одной строфе?

```
SELECT ?герой WHERE {
  ?герой a ?класс .
  ?класс rdfs:subClassOf :Персонаж .
  FILTER NOT EXISTS { ?герой :упомянут ?с }
}
```

Ответ: `:Автор`, `:Дядя`, `:Ларина`.

Кто упомянут в главе 6 и не упомянут в главе 8?

```
SELECT DISTINCT ?герой WHERE {
  ?герой :упомянут ?с . ?с :глава 6 .
  MINUS { ?герой :упомянут ?с8 . ?с8 :глава 8 }
}
```

Ответ: `:Зарецкий`, `:Ольга`, `:Гильо`.

`FILTER NOT EXISTS` оставляет решения, для которых внутренний шаблон не находится. `MINUS` вычитает решения с общими переменными. В простых случаях они дают одно и то же.

Ответ говорит о графе. Ларина в романе упомянута. Правило извлечения её не ищет, поэтому тройки «Ларина упомянута» в графе нет.

Это **открытый мир** из лекции 2: отсутствие тройки значит «неизвестно».

Главное

Запрос с отсутствием отвечает на вопрос «чего нет в графе». Перенести ответ на мир можно, только если граф полон по этому свойству.

Счёт: COUNT, GROUP BY, HAVING

```
SELECT ?герой (COUNT(?c) AS ?строф) WHERE {
    ?герой :упомянут ?c .
} GROUP BY ?герой ORDER BY DESC(?строф)
```

герой	строф
:Татьяна	102
:Онегин	92
:Ленский	43
:Ольга	28
:Филипьевна	13
:Зарецкий	7
:Гильо, :Князь	по 2

Главы, где Ленский упомянут в пяти строфах и больше

```
SELECT ?глава (COUNT(?c) AS ?строф) WHERE {
    :Ленский :упомянут ?c . ?c :глава ?глава .
} GROUP BY ?глава
  HAVING (COUNT(?c) >= 5) ORDER BY ?глава
```

глава 2	глава 3	глава 4	глава 5	глава 6
5	5	6	8	14

`GROUP BY` задаёт группы, `COUNT`, `SUM`, `AVG`, `MIN`, `MAX` считают в группе, `HAVING` отбирает группы.

Главное

Татьяна упомянута в большем числе строф, чем Онегин. Счёт по графу отвечает на вопросы о корпусе в целом, на которые выдача поиска не отвечает.

Пути по свойствам: пять операторов

запись	смысл	пример на «Онегине»
`p/q`	сначала `p`, затем `q`	`:Татьяна :сестра/^:жених ?x` даёт Ленского
`^p`	`p` в обратную сторону	`:гибель_Ленского ^:привелоК ?x` даёт дуэль
`p|q`	`p` или `q`	`:Татьяна (:сестра|:мать) ?x` даёт Ольгу и Ларину
`p+`	один шаг `p` или больше	`:именины :привелоК+ ?x` даёт всю цепочку
`p*`	ноль шагов или больше	`?e a/rdfs:subClassOf* :Событие` даёт все события

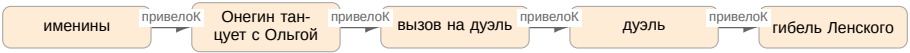
Путь стоит на месте предиката и заменяет цепочку троек с промежуточными переменными. Операторы комбинируются: `(:сестра|:мать)/:живёт`.

Путь отвечает на вопрос «достижим ли узел». Саму цепочку рёбер путь не возвращает: для неё пишут шаблон с промежуточными переменными.

Главное

Пути по свойствам дают запросы на связи произвольной длины. Ни поиск по словам, ни векторный поиск такого запроса не выражают.

Цепочка причин: мини-практика курса



К чему привели именины?

```
SELECT ?следствие WHERE {
    :именины :привелоК+ ?следствие }
```

ссора на балу, вызов, дуэль, гибель Ленского

Что привело к гибели Ленского?

```
SELECT ?причина WHERE {
    :гибель_Ленского ^:привелоК+ ?причина }
```

дуэль, вызов, ссора на балу, именины

В графе записаны только связи соседних событий. Оператор + проходит цепочку любой длины.

Программа курса формулирует мини-практику так: «все леммы, от которых транзитивно зависит теорема X». Здесь тот же запрос на событиях романа.

Поиск по словам на вопрос «что привело к гибели Ленского» даёт строфы 6.III, 6.XXVIII, 8.III, 4.XII, 8.XLV. Ни одна не называет причины.

Главное

Транзитивное замыкание одним оператором. Цепочка зависимостей, причин или ссылок запрашивается так же.

Последовательность, альтернатива, порядок событий

Жених сестры Татьяны одним путём

```
SELECT ?x WHERE { :Татьяна :сестра/^:жених ?x }
```

Ответ: `:Ленский`.

Ближняя родня Татьяны

```
SELECT ?x WHERE {
  :Татьяна (:сестра|:мать|:муж|:няня) ?x }
```

Ответ: `:Ольга`, `:Ларина`, `:Князь`, `:Филипьевна`.

Все события до письма Онегина

```
SELECT ?раньше WHERE {
  :письмо_Онегина :следует+ ?раньше }
```

встреча в Петербурге, приезд в Москву, гибель Ленского, дуэль, вызов, ссора на балу, именины, сон Татьяны, объяснение в саду, письмо Татьяны: десять событий.

Свойство `:следует` связывает событие с предыдущим. Путь `:следует+` даёт всё, что было раньше.

Главное

Шаблон из двух троек с переменной посередине и путь через косую черту дают один ответ. Путь короче, шаблон умеет вернуть промежуточный узел.

Классы и подсвойства без машины вывода

```
SELECT (COUNT(?e) AS ?событий) WHERE {
    ?e a :Событие }
```

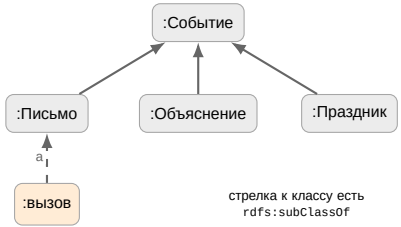
Ответ: **5**.

```
SELECT ?событие ?класс WHERE {
  ?событие a ?класс .
  ?класс rdfs:subClassOf* :Событие . }
```

Ответ: **12** событий: 5 событий, 3 письма, 2 объяснения, 2 праздника.

```
SELECT ?связь ?кто WHERE {
  ?связь rdfs:subPropertyOf :родня .
  :Татьяна ?связь ?кто . }
```

Ответ: мать Ларина, муж Князь, сестра Ольга.



В графе записано: вызов есть письмо, письмо есть подкласс события. Тройки «вызов есть событие» в графе нет. Машина запросов выводов не делает.

Путь `a/rdfs:subClassOf*` проходит иерархию сам. Хранилища умеют вывод по RDFS и OWL как режим работы.

Главное

Запрос видит только записанные тройки. Иерархия классов и свойств из онтологии учитывается путём в запросе или выводом в хранилище.

Окрестность на два шага

```
SELECT ?старт (COUNT(DISTINCT ?сосед) AS ?соседей)
       (GROUP_CONCAT(DISTINCT STRAFTER(STR(?сосед), "#");
                     separator=", ") AS ?кто)
WHERE {
  VALUES ?старт { :Зарецкий :Филипьевна }
  GRAPH g:relations {
    { ?старт ?p1 ?через } UNION { ?через ?p1 ?старт }
    { ?через ?p2 ?сосед } UNION { ?сосед ?p2 ?через }
  }
  ?сосед a ?класс . ?класс rdfs:subClassOf :Персонаж .
  FILTER(?p1 != rdf:type && ?p2 != rdf:type
         && ?сосед != ?старт)
} GROUP BY ?старт
```

старт	соседей	кто
:Зарецкий	4	Ольга, Онегин, Ленский, Гильо
:Филипьевна	3	Ларина, Князь, Ольга

Мини-практика лекции 7: окрестность сущности на два шага.

- `VALUES` задаёт начальные узлы списком;
- `GRAPH g:relations` ограничивает обход ручной разметкой;
- два `UNION` разрешают идти по ребру в любую сторону;
- `GROUP_CONCAT` собирает имена в одну строку.

За два шага от Зарецкого лежит невеста Ленского. В строфах о Зарецком имени Ольги нет.

Главное

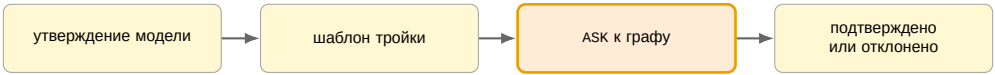
Окрестность узла есть контекст, который граф отдаёт агенту о сущности из вопроса.

ASK: проверка утверждения

запрос	ответ
`ASK { :Онегин :друг :Ленский }`	true
`ASK { :Ленский :жених :Татьяна }`	false
`ASK { :Ленский :жених :Ольга }`	true
`ASK { :Зарецкий :секундант :Онегин }`	false

ASK отвечает «да» или «нет»: есть ли у шаблона хотя бы одно решение.

Языковая модель написала «Ленский был женихом Татьяны». Утверждение переводится в шаблон, граф отвечает «нет». Модель написала «Зарецкий был секундантом Онегина». Граф отвечает «нет», запрос `?x :секундант :Онегин` называет Гильо.



Главное

ASK есть проверка ответа модели по графу. Этот контур проверки собирается в лекции 8. «Нет» значит «в графе не записано».

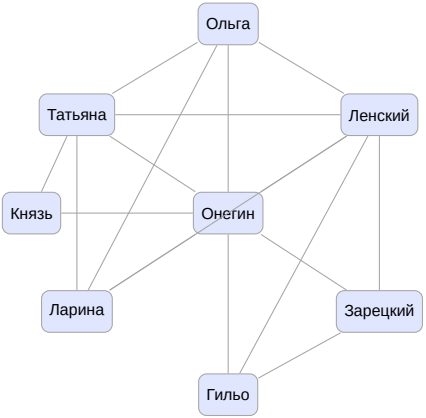
CONSTRUCT: новый граф из старого

```
CONSTRUCT { ?a :знакомы ?b }
WHERE {
    ?событие :участник ?a , ?b .
    FILTER(STR(?a) < STR(?b))
}
```

Ответ `CONSTRUCT` есть граф RDF. Шаблон после `CONSTRUCT` заполняется каждым решением `WHERE`. Здесь два героя знакомы, если участвовали в одном событии. Условие `STR(?a) < STR(?b)` убирает повтор пары в обратном порядке.

Получилось 17 новых троек. Их можно добавить в граф или сохранить в файл.

`DESCRIBE :узел` возвращает описание узла. Его состав определяет хранилище.



17 рёбер «знакомы»

Главное

CONSTRUCT выводит новые связи из старых и записывает их в виде графа. Так строится производный граф для следующего шага конвейера.

Именованные графы и происхождение

```
SELECT ?граф (COUNT(*) AS ?троек) WHERE {
    GRAPH ?граф { ?s ?p ?o }
} GROUP BY ?граф
```

g:relations 269, g:text 2453

Откуда граф знает об Ольге

```
SELECT ?граф ?свойство (COUNT(?o) AS ?значений)
WHERE { GRAPH ?граф { :Ольга ?свойство ?o } }
GROUP BY ?граф ?свойство
```

граф	свойство	значений
g:relations	rdf:type	1
g:relations	rdfs:label	1
g:relations	:мать	1
g:relations	:сестра	1
g:relations	skos:altLabel	1
g:text	:упомянут	28

Набор данных состоит из графов с именами. `GRAPH ?граф` говорит, в каком графе нашлась тройка. Запрос без `GRAPH` здесь видит объединение графов.

Сестра и мать записаны человеком. Упоминания в 28 строфах извлечены программой и могут содержать ошибки извлечения.

Главное

Имя графа хранит происхождение факта. Агент, который ссылается на факт, может назвать и его источник.

Граф и поиск по словам на одном вопросе

вопрос	поиск по словам, первые пять	запрос к графу
ответ Онегина на письмо Татьяны	нужных строф 0 из 5	5 строф через `:ответНа`
что привело к гибели Ленского	ни одна строфа не называет причины	4 события через `^:привелоК+`
кто был секундантом у жениха сестры Татьяны	вопрос без общих слов с ответом	Зарецкий через три ребра
строфы со словом «письмо»	строфы с весом и порядком	`CONTAINS` даёт 18 строф без порядка

SPARQL умеет искать подстроку в тексте строфы. Такой поиск не нормализует слова и не ранжирует выдачу. Индекс занятия 5 делает это лучше.

Граф отвечает на вопросы о связях, если связи записаны. Связь «ответ на» в романе нигде не названа словами, её записал человек. Как записывать связи программой, разбирает лекция 7: связывание сущностей и извлечение троек языковой моделью.

Главное

Каналы делят работу. Поиск по словам находит строфу по её словам, граф находит строфу по связи. Соединить их выдачи предстоит в лекции 8.

Тот же приём на положении о рецензировании

Граф положения о рецензировании из модуля 1 хранит связи «зависит от» между пунктами, разделами и терминами.

От чего зависит пункт 6.5 «Принятие решения» (R65)?

```
PREFIX : <http://example.org/review.owl#>
SELECT ?название WHERE {
    :R65 :зависит_от+ ?x .
    ?x rdfs:label ?название }
```

Без + находятся 4 прямые зависимости: пункты 6.1 «Число рецензий», 6.4 «Ответ на замечания», раздел 6, термин «Решение».

С +: 14 зависимостей

пункты	6.1 «Число рецензий», 6.3 «Содержание рецензии», 6.4 «Ответ на замечания»
разделы	1 «Область применения», 3 «Термины и определения», 6 «Порядок рецензирования»
термины	«Автор», «Замечание», «Ответ авторов», «Редакционная коллегия», «Рецензент», «Рецензия», «Решение», «Статья»

Это вопрос редактора из лекции 1. Поиск по словам тогда нашёл сам пункт 6.5, и пункты 6.1 и 6.4 остались без ответа.

Главное

Запрос пути по свойству одинаков для причин в романе и зависимостей в положении. Меняется только граф.

Точка доступа SPARQL

```
# Oxigraph: один исполняемый файл
oxigraph load --location store \
         --file data/onegin_relations.ttl
oxigraph serve --location store
#   http://localhost:7878/query

# Apache Jena Fuseki: сервер на Java
fuseki-server --file=data/onegin_relations.ttl /onegin
#   http://localhost:3030/onegin/sparql
```

```
import requests
r = requests.get("http://localhost:7878/query",
    params={"query": PREFIXES +
        "SELECT ?кто WHERE { ?кто :сестра :Татьяна }"},
    headers={"Accept": "application/sparql-results+json"})
r.json()["results"]["bindings"]
```

В блокноте граф лежит в памяти процесса Python, запросы выполняет `rdflib`.

В работе граф хранят в **хранилище троек**. Запросы принимает **точка доступа**: адрес HTTP, куда отправляется текст запроса. Ответ приходит в JSON, CSV или RDF.

Протокол один для всех хранилищ. Код агента не меняется, когда граф переезжает из файла в хранилище.

Главное

Точка доступа превращает граф в службу. К ней обращаются блокнот, агент и сервер MCP лекции 11.

SPARQL как инструмент агента

```
{"name": "sparql_query",
 "description": "Запрос SPARQL 1.1 к графу «Евгения
   Онегина». Префикс : уже задан. Классы: :Персонаж,
   :Событие, :Строфа. Свойства: :сестра, :жених,
   :друг, :муж, :участник, :привелоК, :ответНа,
   :вСтрофах, :упомянут. Возвращает не больше 20 строк.",
 "parameters": {"query": "string"}}
```

```
-> SELECT ?причина WHERE { :гибель_Ленского ^:привелоК+ ?причина }
<- {"строк": 4, "строки": [{"причина": ":дуэль"},
    {"причина": ":вызов"}, {"причина": ":ссора_на_балу"},
    {"причина": ":именины"}]}
-> SELECT ?x WHERE { :Ленский :жених ?x
<- {"ошибка": "Expected {SelectQuery | ... found end of text"}
```

Описание инструмента есть контекст модели. В нём словарь графа: классы и свойства. Без словаря модель выдумывает имена свойств.

Инструмент ограничивает число строк, чтобы ответ не переполнил контекст. Синтаксическая ошибка возвращается модели текстом, и модель исправляет запрос на следующем шаге цикла.

Протокол вызова есть доказательство ответа: запрос и строки, которые он вернул.

Главное

Граф входит в агента как инструмент с описанием словаря. Ответ агента ссылается на запрос, и запрос можно выполнить заново.

Типичные ошибки

что написано	что происходит	как исправить
`"Таня"` без метки языка	пустой ответ: в графе `"Таня"@ru`	`"Таня"@ru` или `FILTER(STR(?x) = "Таня")`
`?e a :Событие`	5 событий из 12: подклассы не видны	`?e a/rdfs:subClassOf* :Событие`
ребро в обратную сторону	пустой ответ	`^p` или `UNION` двух направлений
переменная в ответе без `GROUP BY`	ошибка или случайное значение	добавить в `GROUP BY` или в агрегат
обязательная тройка для редкого свойства	пропали строки без свойства	`OPTIONAL`
пустой ответ `FILTER NOT EXISTS` или `ASK`	«в графе не записано»	проверить полноту графа по свойству
забытый `PREFIX`	ошибка разбора	префиксы в начале запроса

Главное

Почти все пустые ответы объясняются несовпадением записи: язык литерала, направление ребра, класс без подклассов.

Шпаргалка

```
PREFIX : <http://example.org/onegin#>
SELECT DISTINCT ?x ?y (COUNT(?z) AS ?n)    # форма ответа: SELECT, ASK, CONSTRUCT, DESCRIBE
WHERE {
  ?x :p ?y ; :q ?z .                        # шаблон: ; тот же субъект, , тот же предикат
  ?y a/rdfs:subClassOf* :Класс .            # путь: / ^ | + *
  OPTIONAL { ?x :r ?w }                     # необязательная часть
  { ?x :s ?v } UNION { ?v :s ?x }           # альтернатива
  FILTER(?n > 2 && LANG(?l) = "ru")         # условие
  FILTER NOT EXISTS { ?x :t ?u }            # отсутствие в графе
  BIND(STR(?l) AS ?строка)                  # новая переменная
  VALUES ?x { :a :b }                       # список значений
  GRAPH ?g { ?x :p ?y }                     # именованный граф
}
GROUP BY ?x ?y HAVING (COUNT(?z) > 1)       # группы и условие на группу
ORDER BY DESC(?n) LIMIT 10 OFFSET 0         # порядок и страница ответа
```

Главное

Шаблон, фильтр, путь, группа. Эти четыре понятия покрывают почти все запросы агента к графу.

Упражнения

Ответы в конце блокнота `sparql_tutorial.ipynb`.

1. Кто участвовал в дуэли? Выведите метки участников.
2. Найдите все письма вместе с автором и адресатом.
3. Какие события произошли в Петербурге? Отсортируйте по главе.
4. Сколько строф каждой главы упоминают Татьяну?
5. В каких строфах упомянуты одновременно Татьяна и Ольга?
6. Какие события прямо или через цепочку привели к последнему объяснению Татьяны?
7. Проверьте запросом `ASK` утверждение «Зарецкий был секундантом Онегина».
8. Постройте запросом `CONSTRUCT` связь «соседи»: два персонажа живут в одном месте.

На своём корпусе. Постройте граф своей области из модуля 1 и напишите запрос пути «все утверждения, от которых транзитивно зависит утверждение X».

Главное

Упражнения 1–5 закрепляют шаблоны и счёт, 6–8 пути и другие формы ответа.

Итоги

Что введено

- тройка, IRI, литерал, запись Turtle;
- шаблон графа, переменная, соединение;
- `FILTER`, `OPTIONAL`, `UNION`, `BIND`, `VALUES`;
- отсутствие и открытый мир;
- агрегаты и группы;
- пути по свойствам;
- `ASK`, `CONSTRUCT`, именованные графы;
- точка доступа и инструмент агента.

Где пригодится

- **лекция 7**: граф как канал поиска, связывание сущностей, окрестность узла;
- **лекция 8**: проверка ответа модели запросом `ASK`, слияние выдач графа и поиска;
- **лекция 9**: агент выбирает между поиском по словам, векторами и графом;
- **лекция 11**: сервер MCP над графом знаний.

Главное

Граф отвечает на вопросы о связях. SPARQL есть способ задать такой вопрос точно и получить ответ, который можно проверить.

Литература и инструменты

1. Harris S., Seaborne A. (ред.) SPARQL 1.1 Query Language. Рекомендация W3C, 2013. `w3.org/TR/sparql11-query`
2. Cyganiak R., Wood D., Lanthaler M. (ред.) RDF 1.1 Concepts and Abstract Syntax. Рекомендация W3C, 2014.
3. Beckett D., Berners-Lee T., Prud'hommeaux E., Carothers G. RDF 1.1 Turtle. Рекомендация W3C, 2014.
4. DuCharme B. Learning SPARQL. 2-е изд. O'Reilly, 2013.
5. Hogan A. и др. Knowledge Graphs. ACM Computing Surveys, 54(4), 2021.
6. rdflib: библиотека RDF и SPARQL для Python. `rdflib.readthedocs.io`
7. Oxigraph: хранилище троек и точка доступа. `github.com/oxigraph/oxigraph`
8. Apache Jena Fuseki: сервер SPARQL. `jena.apache.org`